SOFTWARE REQUIREMENTS SPECIFICATION

CampusMind AI

University Knowledge Assistant using RAG,
Hybrid Search, and Redis Answer Caching

Document control	Value
Document	Software Requirements Specification
Version	1.3
Project	CampusMind AI
Status	Updated implementation baseline
Supersedes	Version 1.2

Implementation principle: Prioritize the required RAG, hybrid retrieval, indexing, caching, and evaluation work. Keep the solution simple, measurable, and easy to explain. Defer non-core interface work until the technical pipeline is working.

Prepared as the functional and technical baseline for implementation.

Document Control and Change Summary

Version	Date	Summary
1.2	November 2026	Initial requirements baseline.
1.3	November 2026	Reordered scope around the core technical evaluation; clarified chunking default and evaluation; made frontend/authentication lower priority; clarified retrieval, cache, and test acceptance criteria.

Contents

- 1. Introduction and scope
- 2. Priorities and delivery phases
- 3. Technology stack
- 4. System architecture and core workflows
- 5. Document ingestion and chunking
- 6. Embeddings, storage, and indexing
- 7. Retrieval and answer generation
- 8. Redis answer caching
- 9. Data storage and metadata
- 10. Functional requirements
- 11. Non-functional requirements
- 12. Testing and evaluation
- 13. Minimal API and later interface scope
- 14. Project structure and deliverables
- 15. Constraints and development guidelines

1. Introduction and Scope

1.1 Purpose

This SRS defines the requirements for CampusMind AI, a university knowledge assistant. Users ask questions about university documents and receive answers grounded in retrieved passages, with references to the source documents.

1.2 Project overview

The system extracts text from uploaded PDF, DOCX, and TXT documents; splits the text into metadata-preserving chunks; creates embeddings; and stores vectors in Qdrant. It retrieves evidence using both Qdrant vector search and BM25 keyword search, combines rankings with Reciprocal Rank Fusion (RRF), and sends selected evidence to an external LLM. Redis caches successful answers and their source references so repeated questions can bypass embedding, retrieval, and generation.

1.3 Main objective

Build and evaluate a working, explainable RAG pipeline. The primary demonstration must prove document ingestion, chunking, vector and keyword retrieval, hybrid ranking, grounded answers with citations, and Redis cache hits that avoid repeated expensive work.

1.4 Scope

- **Core scope:** document extraction; fixed-size, recursive, and semantic chunking with configurable overlap; embeddings; Qdrant HNSW vector search; BM25; RRF; evidence checking; LLM answer generation with citations; Redis cache-aside, invalidation, and metrics; and technical evaluation.
- **Supporting scope:** MySQL records for documents and chat interactions; a small FastAPI interface for upload, processing, and question-answering.
- **Deferred scope:** polished React pages, dashboards, advanced authentication flows, and UI styling. These must not delay the core technical work.
- **Out of scope initially:** microservices, distributed task queues, multi-agent workflows, model training, OCR for scanned PDFs, and advanced reranking.

Priority rule: A minimal API or test client is sufficient to demonstrate the core pipeline. Complete and test the core technical requirements before spending significant time on login, dashboard, or frontend presentation.

2. Priorities and Delivery Phases

Priority	Work	Completion condition
P0 – Main evaluation focus	Ingestion, all chunking methods, embeddings, Qdrant/HNSW, BM25, RRF, Redis cache, grounded LLM answers, citations, evaluation.	End-to-end RAG works and each core feature has test evidence.
P1 – Supporting backend	FastAPI endpoints, document metadata, chat interaction storage, basic error handling and configuration.	Core pipeline can be demonstrated through API/Postman or a minimal client.
P2 – Later presentation work	JWT login, role-based UI, React pages, admin dashboard, cache analytics screen, styling.	Implement after P0 is stable and tested.

The project remains one modular FastAPI backend. Deferring interface work does not remove it from the eventual product scope; it controls implementation order.

3. Technology Stack

Area	Technology	Use
Backend/API	Python + FastAPI	One modular application; expose ingestion and question-answering endpoints.
Document extraction	PyMuPDF, python-docx, plain text	Extract text from PDF, DOCX, and TXT.
Chunking	LangChain Text Splitters + simple custom Python	Recursive and fixed-size splitting; a simple embedding-based semantic splitter.
Embeddings	Sentence Transformers: BAAI/bge-small-en-v1.5	Embed document chunks and user questions consistently.
Vector database	Qdrant	Store chunk vectors and metadata; perform vector similarity search.
Vector index	Qdrant HNSW	Approximate nearest-neighbor indexing for vector retrieval.
Keyword retrieval	BM25	Retrieve chunks using exact terms, names, dates, codes, and identifiers.
Hybrid fusion	Reciprocal Rank Fusion (RRF)	Combine vector and BM25 ranked lists.
Cache	Redis + redis-py	Cache successful answers and source references.
LLM	External LLM API	Generate answers only from retrieved evidence.
Relational database	MySQL	Store document records and chat interactions.
Local services	Docker Desktop	Run Qdrant and Redis locally.
Testing/tools	pytest, Postman, Git/GitHub, MySQL Workbench	Automated checks, API testing, source control, and database inspection.
Later frontend	React + Tailwind CSS	User-facing pages after the core pipeline is complete.

4. System Architecture and Core Workflows

4.1 Main components

- **Ingestion service:** validates files, extracts text, cleans it, chunks it, and attaches source metadata.
- **Embedding service:** uses the same embedding model for document chunks and incoming questions.
- **Vector store:** Qdrant stores vectors and payloads and performs HNSW-backed similarity search.
- **Keyword index:** BM25 indexes the same chunk collection used by vector retrieval.
- **Retrieval service:** runs vector and keyword retrieval and merges rankings with RRF.
- **Answer service:** checks evidence, calls the configured LLM, and returns an answer with source references.
- **Cache service:** checks Redis before retrieval and stores only successful grounded answers.
- **FastAPI:** coordinates the services and exposes a small testable API.

4.2 Main question-answering flow

Question → normalize and create cache key → Redis check → cache hit: return answer + sources; cache miss: embed question → Qdrant vector search + BM25 → RRF → evidence check → LLM answer + citations → Redis cache → response.

Redis lookup must happen before question embedding, Qdrant, BM25, RRF, or LLM generation. A cache hit skips all of those steps.

4.3 Main ingestion flow

Upload → validate → extract and clean → run selected chunking method → attach metadata → embed chunks → store vectors and payloads in Qdrant → update BM25 index → update document status and knowledge-base version.

5. Document Ingestion and Chunking

5.1 Supported documents

The initial version supports text-based PDF, DOCX, and TXT files. Validate file type and configured size limits. Scanned PDFs requiring OCR are outside the initial scope.

5.2 Required chunking methods

Method	Required behavior	Role in evaluation
Fixed-size	Split text into configured windows with configurable overlap.	Simple baseline; may cut across sentence or paragraph boundaries.
Recursive	Split using progressively smaller separators to preserve paragraphs and sentences where possible; configurable overlap.	Recommended initial default because it balances simplicity and preservation of document structure.
Semantic	Split at meaningful changes in content using sentence embeddings/similarity and a configurable threshold; retain overlap between adjacent chunks.	Compare whether meaning-aware boundaries improve retrieval enough to justify added processing.

5.3 Initial default and configuration

Setting	Initial value
Default method	Recursive character text splitting
Target chunk size	Approximately 500 tokens
Overlap	Approximately 75 tokens
Adjustment	Tune after inspecting actual document chunks and evaluation results.

Implementation note: if the chosen splitter measures size in characters, do not label a character count as tokens. Use a tokenizer-based length function where practical, or document the character approximation clearly. Keep chunks within the embedding model's supported input length; avoid silent truncation.

5.4 Chunk metadata

Every chunk must retain enough metadata to trace a retrieved passage back to its origin:

- documentId and documentName
- chunkId and chunkIndex
- chunkingMethod
- pageNumber when available
- chunk text and any required access-scope metadata

The same chunk identity and source metadata must be preserved in Qdrant, BM25 results, RRF output, and final citations.

5.5 Chunking decision

All three methods remain implemented and configurable. Recursive is the starting default, not a predetermined winner. Compare all methods on the same corpus, embedding model, retrieval settings, and question set. Choose the final default using measured retrieval quality, answer/citation quality, and processing cost.

6. Embeddings, Storage, and Indexing

6.1 Embeddings

Use BAAI/bge-small-en-v1.5 for both document chunks and questions. The vector dimension configured in Qdrant must match the model output. Apply the model's recommended query/document input conventions consistently.

6.2 Qdrant

- Store one point per chunk, with a unique point ID, embedding vector, and payload containing chunk text and metadata.
- Use cosine similarity and Qdrant's HNSW index with reasonable defaults.
- Create payload indexes only for fields actually used in filters, such as documentId or department if department filtering is implemented.
- Use documentId to remove or replace all points belonging to a document during update, delete, or reindex.

6.3 BM25 index

Build a BM25 keyword index over the same chunk corpus. Preserve each result's chunk ID and metadata so it can be fused with Qdrant results. Keep the BM25 index synchronized when documents are added, updated, deleted, or reprocessed. Persist/rebuild the index in a simple reliable way appropriate to the project; do not add a separate search service.

7. Retrieval and Answer Generation

7.1 Hybrid retrieval

- Run Qdrant vector search and BM25 keyword search for each cache miss.
- Retrieve ranked candidate lists from both methods.
- Combine rankings using Reciprocal Rank Fusion (RRF), using a configurable candidate count and RRF constant.
- Deduplicate by chunk identity and preserve source metadata.
- Return the best evidence chunks to the answer-generation step.

7.2 Evidence and answer rules

- The LLM must receive the user question and retrieved document passages, not an unrestricted request to answer from general knowledge.
- The answer must include source references that identify the document and page or chunk where available.
- If retrieved evidence is insufficient, return a clear response that the answer was not found in the available university documents.
- Do not invent citations or unsupported facts. Do not cache insufficient-evidence, failed, or ungrounded responses as successful answers.

7.3 Provider configuration

Keep the LLM provider, model, API key, and generation settings configurable through environment variables. The LLM provider is used only for final answer generation; it must not be tightly coupled to chunking, embeddings, BM25, Qdrant, RRF, or Redis.

8. Redis Answer Caching

8.1 Required cache-aside behavior

- Normalize the question and construct a deterministic cache key.

- Check Redis before query embedding or any retrieval/generation work.
- On a hit, return the cached answer and source references and skip embedding, Qdrant, BM25, RRF, and LLM calls.
- On a miss, run the normal RAG pipeline; cache only a successful grounded answer and its source references.
- Save the current user's chat interaction in MySQL on both cache hits and misses.

8.2 Cache key, TTL, and invalidation

Use a key containing a namespace, knowledge-base version, applicable access scope, and SHA-256 hash of the normalized question. Initial TTL: 1,800 seconds (30 minutes), configurable through environment settings. Increment the knowledge-base version or invalidate relevant entries whenever documents are added, updated, deleted, or reindexed. Cache keys must prevent answers based on restricted documents from being shared across unauthorized access scopes.

8.3 Reliability and metrics

If Redis is unavailable, log the cache failure and continue through the normal RAG pipeline where possible. Redis is an optimization, not the source of truth. Record hits, misses, hit rate, response latency, and LLM calls avoided; record memory usage if available. Never claim performance gains without measuring them.

9. Data Storage and Metadata

9.1 MySQL

Table	Initial fields / purpose
documents	id, filename, file_path, uploader_id if available, upload_date, status, chunk_count, version
chat_sessions	id, user_id where authentication is enabled, title, created_at
messages	id, session_id, role, content, timestamp, source references, cache-hit indicator

User/authentication tables can be added in the later authentication phase. Add indexes to frequently queried or joined columns based on actual queries; do not index every column.

9.2 Source of truth

Original uploaded files and document records remain the source of truth. Qdrant vectors, BM25 entries, and Redis answers are derived data and must be rebuildable or invalidatable when source documents change.

10. Functional Requirements

ID	Requirement
FR-01	Accept supported PDF, DOCX, and TXT uploads and reject unsupported or invalid files.
FR-02	Extract and clean document text while preserving page information when available.
FR-03	Implement fixed-size, recursive, and semantic chunking; each supports configurable overlap.
FR-04	Attach document identity, chunk identity/index, method, and page metadata where available to every chunk.
FR-05	Generate chunk embeddings using BAAI/bge-small-en-v1.5.
FR-06	Store vectors and chunk payloads in Qdrant using cosine similarity and HNSW indexing.
FR-07	Maintain a BM25 keyword index over the same chunks and synchronize it with document lifecycle changes.
FR-08	Run vector and BM25 retrieval and combine ranked results using RRF.
FR-09	Check retrieved evidence and avoid unsupported answers when evidence is insufficient.
FR-10	Generate answers from retrieved context and return accurate source references.
FR-11	Check Redis before embedding/retrieval/generation and return cached answer plus sources on a hit.
FR-12	Cache only successful grounded answers and source references on a miss.
FR-13	Use configurable TTL, version-aware cache keys, and invalidation after document changes.
FR-14	Continue normal RAG operation when Redis is unavailable, where possible.
FR-15	Record cache hits, misses, latency, hit rate, and LLM calls avoided.
FR-16	Store document records and chat interactions in MySQL; save interactions for cache hits too.
FR-17	Expose minimal FastAPI endpoints to upload/process documents and submit questions for testing.
FR-18	Later phase: implement JWT authentication, role enforcement, React pages, and admin views.

11. Non-Functional Requirements

Category	Requirement
Answer quality	Answers must be grounded in retrieved university content and provide traceable citations.
Performance	A cache hit must bypass embedding, retrieval, fusion, and LLM generation. Measure latency rather than assuming improvement.
Reliability	Redis failure must not unnecessarily make the RAG system unavailable.
Consistency	Document updates/deletions must synchronize Qdrant, BM25, and cache version/invalidation.
Maintainability	Use a single modular backend with readable, explainable code and limited dependencies.
Security	Keep API keys and credentials in environment variables; do not commit secrets. Enforce authorization before exposing restricted documents.
Privacy	Respect document access scope in retrieval, citations, and cache keys.
Scalability	Use HNSW and Redis appropriately; avoid premature infrastructure complexity.

12. Testing and Evaluation

12.1 Core functional tests

- Upload and extraction tests for PDF, DOCX, TXT, invalid files, and empty documents.
- Chunking tests for all three methods: overlap, non-empty chunks, metadata preservation, and source traceability.
- Embedding/Qdrant tests: vector dimensions, point storage, similarity search, and document deletion/replacement.
- BM25 tests for exact terms, names, dates, and identifiers; verify index synchronization.
- RRF tests for ranking combination, deduplication, and preservation of chunk metadata.
- Answer tests for grounded responses, correct citations, and insufficient-evidence behavior.
- Redis tests for miss, hit, TTL expiry, invalidation, access-scope isolation, and Redis-unavailable fallback.
- Verify with instrumentation or mocks that a cache hit makes no embedding, Qdrant, BM25, RRF, or LLM call.

12.2 Chunking comparison

Use the same document corpus, embedding model, retrieval settings, and fixed set of approximately 20–30 questions for all methods. Disable Redis during this comparison. Include questions answerable from one document, questions needing multiple documents, exact names/dates/identifiers, questions absent from the corpus, and unrelated questions.

Compare chunk count and size, processing time, retrieval quality (Recall@k and MRR where feasible), answer correctness, and citation accuracy. Record results in a table. Select the best-performing method as the normal-ingestion default; keep all three methods available.

12.3 Cache comparison

Run the same question set with caching disabled and enabled. Report measured response latency, hit rate, and LLM calls avoided. Test repeated questions and verify that cache hits bypass the entire expensive pipeline. Hit rate = hits / (hits + misses) × 100%.

12.4 Acceptance criteria for the main demonstration

- At least one document can be ingested end-to-end and its chunks can be traced to the original source.
- Both vector and BM25 retrieval return relevant chunk candidates; RRF combines them.
- A question receives an evidence-grounded answer with correct source references.
- An unsupported question receives a clear not-found response and is not cached as a successful answer.
- Repeating a successful question produces a Redis hit and avoids embedding, retrieval, and LLM generation.
- Measured chunking and cache comparison results are documented; no invented performance claims.

13. Minimal API and Later Interface Scope

13.1 Initial API

Only implement the endpoints needed to test the core work, for example:

- **POST /documents/upload** – upload and process a supported document.
- **GET /documents** – list processed document records (basic implementation).
- **POST /chat/ask** – submit a question and receive an answer, citations, and cache-hit status.
- **GET /health** – basic application health check.

Endpoint names may be adjusted to fit the implementation. Keep the API small; do not build a large authentication or admin API before the core pipeline works.

13.2 Later UI/authentication scope

The eventual interface may contain the six pages defined in v1.2: Home, Login/Register, AI Chat, Admin Dashboard, Document Management, and Redis Cache Analytics. JWT authentication and backend role-based access remain required before production or use with private documents. These are later-phase tasks and must not displace the core technical evaluation.

14. Project Structure and Deliverables

Suggested initial structure (create folders only when needed):

```
campusmind-ai/
■■■ backend/
■ ■■■ app/
■ ■ ■■■ main.py
■ ■ ■■■ api/
■ ■ ■■■ core/
■ ■ ■■■ services/
■ ■ ■ ■■■ ingestion/
■ ■ ■ ■■■ chunking/
■ ■ ■ ■■■ embeddings/
■ ■ ■ ■■■ retrieval/
■ ■ ■ ■■■ cache/
■ ■ ■ ■■■ generation/
■ ■ ■■■ repositories/
■ ■ ■■■ schemas/
■ ■ ■■■ models/
■ ■■■ tests/
■ ■■■ requirements.txt
■■■ storage/
■ ■■■ uploads/
■■■ docker-compose.yml
■■■ .env.example
■■■ .gitignore
■■■ README.md
```

- Working FastAPI RAG backend implementing the core pipeline.
- Three overlap-enabled chunking methods and a documented comparison.
- Qdrant collection with vectors, metadata, HNSW search, and useful payload indexes.
- BM25 keyword index and RRF hybrid retrieval.
- Redis answer caching with TTL, invalidation, fallback, and measured metrics.
- Grounded answers with accurate source references and insufficient-evidence handling.
- MySQL document/chat records as needed for the working backend.
- Tests, README setup instructions, architecture overview, environment configuration, and measured evaluation results.
- Later: React UI, authentication, role restrictions, and admin analytics pages.

15. Constraints and Development Guidelines

- Implement one core feature at a time and test it before moving on.
- Keep code simple, modular, clean, and easy to explain in a live presentation.
- Do not add unnecessary services, dependencies, abstractions, or advanced infrastructure.
- Do not start frontend polish, dashboard work, or elaborate login flows while core RAG requirements remain incomplete.
- Keep BM25 and RRF as required parts of retrieval; do not replace hybrid retrieval with vector-only search.
- Keep all three chunking methods and configurable overlap; recursive is only the initial default until evaluation.
- Preserve source metadata throughout ingestion, retrieval, generation, and response formatting.
- Measure retrieval quality and cache performance. Report actual results only.
- Keep secrets out of source control and use `.env.example` for documented configuration names.
- Treat document files as the source of truth; rebuild or invalidate derived indexes and cached answers after document changes.

End of SRS — Version 1.3